

Michael Fox

CMPE 172

27, March 2026

Term Project M4

Identify a Race Condition:

A race condition is when there are multiple processes or threads, i.e. concurrent programming, in which information is shared and as such the outcome of multiple threads/processes can depend on the order in which they execute commands in the shared resource.

For example, in my Term Project a situation where a race condition could occur is in scheduling an appointment. To schedule an appointment a user must check the data base to see if a particular time slot is available, if it is available they are able to start the process of booking an appointment. Therefore if I had two concurrent users who both check if a timeslot is available at the same time there is a change it could go something like this:

Sudo code:

Read (time slot availability)

If (available):

Start booking process....

As such if both threads are doing this at the same time we could end up with:

T1 – read(time slot availability) → DB says it is available

T2 - read(time slot availability) → DB says it is available

T1 – if(available)→ evaluates to TRUE

T2 – if(available)→ evaluates to TRUE

T1 – starts booking process

T2 – starts booking process

As there was no lock on the shared data the two threads were both able to start booking the same timeslot thus creating a conflict. To avoid this, I will use the ACID properties:

A – Atomicity: I will make it so if any part of the @transaction fails, the whole transaction will fail leading to a rollback of db.

C – Consistency: The business rule of one appointment per timeslot will be maintained in the system by using a unique constraint appointment.timeslotID

I – Isolation: Concurrent requests will not both be able to claim the same timeslot.

D – Durability: I will utilize MySQL to commit the booked appointments and have them persist in the DB.

Concurrency Control Strategy: I will use a pessimistic control at the row level using the SQL statement in a data source layer function called claimSlotIfAvailable():

```
UPDATE timeSlot
```

```
SET status = 'Booked'
```

```
WHERE timeSlotID = ? AND status = 'Available'
```

This means a timeslot status will change to booked only if it is available. If 1 row is updated, it will indicate that the statement was successful. 0 rows updated if the statement failed. The function will return 0/1. I will also use @Transaction(isolation = Isolation.READ_COMMITTED) in the java code for the function bookAppointment(BookAppointmentRequest Request).

I am setting the isolation level to read committed because it is faster than serializable and is used in conjunction with the atomic conditional update. The transaction boundary in everything inside the bookAppointment() function which will exist within the service layer. By having the sql statement with in the bookAppointment transactions it prevents inconsistencies because if the slot is set to booked, but saving the appointment fails for some reason the slot will roll back to being available because it is with the transaction.

Code Snippet of the @Transactional – bookAppointment():

```
@Service
public class AppointmentService {

    private final TimeSlotRepository timeSlotRepository; 3 usages
    private final AppointmentRepository appointmentRepository; 2 usages

    public AppointmentService(TimeSlotRepository timeSlotRepository, no usages
                              AppointmentRepository appointmentRepository) {
        this.timeSlotRepository = timeSlotRepository;
        this.appointmentRepository = appointmentRepository;
    }

    @Transactional(isolation = Isolation.READ_COMMITTED) no usages 1 related problem
    public Appointment bookAppointment(BookAppointmentRequest request) {

        int updatedRows = timeSlotRepository.claimSlotIfAvailable(request.getSlotId());

        if (updatedRows == 0) {
            throw new SlotUnavailableException("This time slot is no longer available.");
        }

        TimeSlot claimedSlot = timeSlotRepository.findById(request.getSlotId())
            .orElseThrow(() -> new IllegalArgumentException("Time slot not found."));

        Appointment appointment = new Appointment();
        appointment.setTimeSlot(claimedSlot);
        appointment.setPatientId(request.getPatientId());
        appointment.setNotes(request.getNotes());
        appointment.setReasonForAppt(request.getReasonForAppt());

        return appointmentRepository.save(appointment);
    }
}
```